

GPTfy POC1 — Project Walkthrough

Audience: Team / client presentation
Prepared by: Project creator
Date: June 4, 2026
Org: gptfy-poc1 (kesavpoc@gptfy.com)
Checkpoint: Git tag `Part-1`

1. Why this project exists

Client: Tungsten Automation
Product: GPTfy (AI layer on Salesforce)
POC scope: Use Case 2 — **Customer Assist + Knowledge Base Creation via Portal**

The business problem is straightforward:

- Customers wait too long for simple answers.
- Agents spend time on questions that could be self-served.
- When cases are resolved, that knowledge is rarely captured in a reusable Knowledge Base.

What we built (Part 1 — complete and verified):

Goal	How we achieve it
Deflect simple support questions with AI	Guest portal + GPTfy RAG
Close cases automatically when the customer confirms	Platform Event + trigger
Let agents resolve in one click	Resolve quick action
Build a KB over time	Draft Knowledge articles on agent resolve
Email the answer to the customer	Record-triggered email flow

POC products: E-Invoicing, TotalAgility, PowerPDF (3-week POC, \$9,999 investment per signed scope doc).

2. Who uses the system

Person	Role
Guest (portal visitor)	Submits a question, reads the AI answer, confirms or escalates
Support agent	Uses the Resolve button, reviews draft KB articles
Admin / developer	Deploys metadata, configures flows, monitors email

3. The big picture — three paths, one platform

Everything runs on **Salesforce** in org `gptfy-poc1` , with **GPTfy** (`ccai` managed package) for AI.

Path 1 — Guest Portal: Guest on Experience Cloud site → caseResolutionAssistant LWC → GPTfy AI fills Case.Description → Platform Event closes Case.

Path 2 — Agent Resolve: Support Agent → Resolve Quick Action → Draft Knowledge Article created.

Path 3 — Resolution Email: Any case close (guest or agent) → Case_Resolution_Email_RTF → Email to SuppliedEmail.

One rule ties it together: whenever a Case closes with a resolution and a guest email, the customer gets that answer by email.

4. Path 1 — Guest portal (step by step)

This is the **self-service deflection** path. A customer visits the public Experience Cloud site (`gptfysupport`).

Step 1 — Customer fills the form

The UI is the LWC **caseResolutionAssistant**, embedded on the site home page.

Fields:

- First name, last name, email (required)
- Product (optional picklist from `Case.Product__c`)
- Question (required)

The LWC validates input (email regex, required fields) before submit.

Step 2 — Case is created

On **Find Solution**, the LWC calls Apex: `CaseResolutionController.createSupportCase(...)`

This creates a Case with:

- `Origin = 'Web'` (important — the AI flow filters on this)
- `Status = 'New'`
- `Subject` = the question
- `SuppliedName` , `SuppliedEmail` = guest info
- `Product__c` = selected product (optional)

Why without sharing? Guest users on Experience Cloud cannot normally create Cases under default sharing. This controller runs elevated for these three narrow, guest-safe operations only.

Step 3 — AI generates the answer

Case insert fires the record-triggered flow **Case_Resolution_RTF**:

- Filters: `Origin = 'Web'`
- Runs **asynchronously** (HTTP callouts to GPTfy/GCP are not allowed in synchronous DML)
- Invokes GPTfy: `ccai__AIPromptProcessingInvokable` (ExecutePrompt)
- GPTfy RAG searches Knowledge/content and writes the answer to **Case.Description**

The LWC shows a loading spinner and **polls every 2.5 seconds, up to 12 times (~30 seconds)** via `getRecommendation(caseId)` .

When `Description` is populated → show the answer.

If still empty after 30s → show a fallback: *"A support agent will follow up shortly."*

Step 4 — Customer chooses

Two buttons:

Button	What happens
Yes, resolved	Case closes; resolution saved; email sent
No, create a case	Case stays open; customer sees case number for agent follow-up

Step 5 — "Yes, resolved" (the close path)

Guest users **cannot update Cases directly** (Guest User License restriction — not bypassed by without sharing).

So we use a **Platform Event pattern**:

1. LWC calls `resolveCase(caseId)`
2. Apex publishes **Case_Resolved__e** with the Case Id
3. **CaseResolvedTrigger** runs as Automated Process (system context):
 - Sets `Status = 'Closed'`
 - Copies `Description` → `Resolution__c` (what the customer saw)
4. Case update triggers the **email flow** (Path 3 below)

The LWC shows "**Closing your case...**" while this completes (usually 1–2 seconds).

5. Path 2 — Agent Resolve button (step by step)

When AI cannot help, or the customer clicks **No**, the case stays open for an agent.

Step 1 — Agent opens an open Case

On the Case record page (highlights panel), the agent sees a **Resolve** button — only when `Status ≠ Closed` (Dynamic Action visibility filter; one-time Setup step documented in `docs/resolve_button_setup.md`).

Step 2 — Agent clicks Resolve

A modal opens (LWC **caseResolveAction**, wrapped in Aura **caseResolveActionAura** because of a known Salesforce quirk with LWC quick actions on Case).

The agent types the resolution in a textarea.

Step 3 — Agent clicks Save

Apex **CaseResolveActionController.saveResolution(caseld, resolutionText)** does three things:

1. **Updates the Case:** `Resolution__c` = typed text, `Status = 'Closed'`
2. **Creates or updates a Draft Knowledge article:**
 - Title = Case Subject
 - Resolution = agent's text
 - Linked to Case via `CaseArticle`
 - Dedupes: if a draft already exists for this case, it updates instead of creating a duplicate
3. **Returns a result** so the LWC can toast: *"Case resolved. Draft Knowledge article created/updated."*

KB creation is **best-effort** — if it fails, the case still closes and a warning toast is shown.

Step 4 — Email (if guest email exists)

Same email flow as the portal path (Path 3).

Design choice: KB drafting is triggered on **agent resolve**, not on guest self-resolve. Portal-deflected closes copy AI text to `Resolution__c` but do not auto-draft KB articles yet (GPTfy KCS-compliant KB prompt is still pending).

6. Path 3 — Resolution email (automatic)

Flow: **Case_Resolution_Email_RTF** (Active)

When it runs:

- Case is **updated** to `Status = Closed`
- `SuppliedEmail` is not blank
- `Resolution__c` is not blank

What the customer receives:

Field	Source
To	Case.SuppliedEmail
Subject	Case.Subject (their original question)
Body	Intro + decoded resolution text
From	Org-wide email kesavamoorthy@cloudcompliance.app

Technical detail: `Resolution__c` is an HTML field, so quotes can store as `"` . The flow decodes HTML entities before sending so email looks correct.

7. Key Salesforce fields (Case)

Field	Purpose
Subject	Guest question / email subject
Description	AI-generated answer (portal path only)
Resolution__c	Final resolution text (Html field)
SuppliedEmail	Guest email for notifications
SuppliedName	Guest name from form
Origin	web for portal cases
Product__c	Product picklist for KB filtering
Status	Closed triggers email flow

Many other GPTfy-related fields exist on Case (sentiment, RCA, triage, etc.) for **Use Case 1** (Case Intelligence) — provisioned in schema but **not in current focus**.

8. What's in the repo (technical inventory)

```

force-app/main/default/
├─ lwc/
│  └─ caseResolutionAssistant/      ← Guest portal UI (6 UI states)
│     └─ caseResolveAction/        ← Agent resolve modal
├─ aura/
│   └─ caseResolveActionAura/      ← Wrapper for quick action
├─ classes/
│  └─ CaseResolutionController.cls ← Guest Apex (without sharing)
│  └─ CaseResolveActionController.cls ← Agent Apex (with sharing)
│     └─ *Test.cls                 ← Unit tests
├─ triggers/
│   └─ CaseResolvedTrigger.trigger ← Platform event subscriber
├─ objects/
│  └─ Case_Resolved__e/            ← Platform event
│     └─ Case/fields/              ← Custom fields incl. Resolution__c, Product__c
├─ flows/
│  └─ Case_Resolution_RTF.flow      ← GPTfy AI prompt (may need activation)
│     └─ Case_Resolution_Email_RTF.flow ← Resolution email
├─ quickActions/
│   └─ Case.Resolve                ← Agent quick action
├─ permissionsets/
│   └─ gptfysupport_Guest_Access   ← Minimal guest permissions
├─ settings/
│   └─ EmailAuthorization           ← Substitute sender workaround
├─ scripts/apex/                  ← Org verification scripts
├─ docs/PRD.md                   ← Living product doc
└─ POC_CHECKLIST.md              ← Full POC tracker

```

Checkpoint: Git tag **Part-1** marks the verified baseline (2026-06-03).

9. What's done vs. what's still pending

Done (Part 1 — verified in org)

- Guest portal LWC + Apex + polling
- Platform Event close path for guests
- Agent Resolve quick action + KB draft
- Resolution email with plain-text body fix
- Apex unit tests + org verification scripts
- GitHub repo + Part 1 checkpoint

Still pending (for full POC sign-off)

Priority	Item
High	Activate Case_Resolution_RTF flow + verify GPTfy prompt
High	Update Product__c picklist to E-Invoicing, TotalAgility, PowerPDF
High	Import Tungsten DocShield / Knowledge content
Medium	GPTfy KCS-compliant KB drafting prompt (vs. direct copy today)

Medium	Deflection metrics and reports
Medium	UAT + demo recording for leadership

Out of scope for now: Use Case 1 (Case Intelligence) and Use Case 3 (Microsoft Copilot).

10. How to demo this to your team/client

Demo script — Guest path:

1. Open the public Experience Cloud site in incognito
2. Submit a question (pick a product if configured)
3. Wait for AI answer (~5–15 seconds)
4. Click **Yes, resolved**
5. Show: Case closed in Salesforce, email received

Demo script — Agent path:

1. Open an open Case (or one where guest clicked **No**)
2. Click **Resolve** → type resolution → Save
3. Show: Case closed, Draft Knowledge article linked, email sent if `SuppliedEmail` exists

Verification in org:

```
sf apex run --file scripts/apex/testCaseResolutionEmail.apex --target-org gptfy-poc1
```

Test records use subject prefix `cursor-test-{timestamp}` for easy cleanup.

11. One-sentence pitch for leadership

GPTfy POC1 lets customers get instant AI answers on a public portal and close their own cases when satisfied — while agents resolve complex cases in one click and automatically draft Knowledge articles — with every resolution emailed back to the customer.

12. Suggested talking order for your presentation

1. **Business problem** (Section 1) — 2 min
2. **Three paths overview** (Section 3) — 3 min
3. **Live demo: guest portal** (Section 4) — 5 min
4. **Live demo: agent resolve + KB** (Section 5) — 3 min
5. **Email confirmation** (Section 6) — 1 min
6. **What's next** (Section 9) — 2 min

This document is maintained in source control at `docs/GPTfy-POC1-Project-Walkthrough.md` .